

Managing Code Ownership

Martin E. Nordberg III, *Blueprint Technologies*

This article is about ownership—ownership of source code and other software artifacts that contribute to a software system during its development. To speak of code ownership is to trigger a variety of emotions in experienced developers—both disgust for when ownership failed to happen and pride for when it did.

Beyond mere participation, ownership implies

- Direct benefit from increases in value or generated income
- Pride in appearance, achievement, and improvement
- Responsibility for problems, flaws, or routine maintenance
- Commitment to long-term involvement
- Potential for successful resale (reuse)

Applying these general statements to software artifact ownership is straightforward to describe (but difficult to achieve). Developers have specific responsibility for the quality, improvement, and appearance of software system or subsystem artifacts including source code, design models, documentation, and test plans.

I consider four models for software system ownership—that is, models for assigning software systems to their human owners:

1. *Product specialist*: A single individual manages all code with occasional help from other individuals.
2. *Subsystem ownership*: Each subsystem has a specific owner, and each team member owns one or more subsystems.
3. *Chief architect*: A chief programmer (architect) has primary ownership of all code. The team takes supporting roles in fleshing out the team leader's vision.
4. *Collective ownership*: All code is collectively owned. Schedules and responsibilities are set such that every team member has a chance to contribute to every subsystem and is free to work across subsystems as needed.

Over the years, I have seen each model practiced without being labeled, and each worked well under the appropriate circumstances. However, for medium to large projects, I propose that the right model is not one but all four: vary the ownership model over the lifetime of a typical project just as you divide a project into phases—for example, in the Unified Process. In fact, the four ownership models' strengths and weaknesses match the needs of the four Unified Process phases. Furthermore, dynamically assigned ownership can help balance a project's needs for

Code ownership is more than simply assigning subsystems to individuals or subteams. Companies can creatively vary code ownership over the life of a system to meet a software development project's evolving needs.

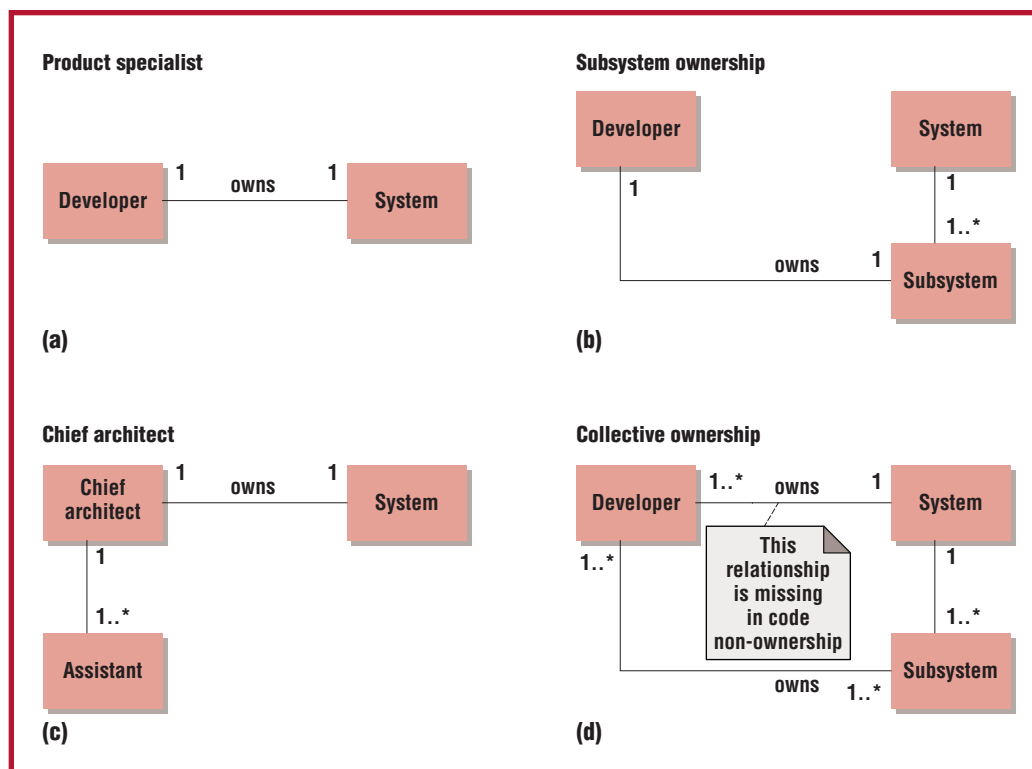


Figure 1. The four models of code ownership: (a) product specialist; (b) subsystem ownership; (c) chief architect; and (d) collective ownership.

communication, productivity, and accountability against its needs for single-minded vision, maintainable product quality, and manageable complexity.

Models of code ownership

Before I discuss the four models in detail, let me explain non-ownership. This is when a system and its subsystems have no owners—everyone changes anything with minimal accountability for system quality or team communication. This is the norm for too many software projects.

Non-ownership's symptoms in software development are

- Poor or missing documentation
- Source code that is unreadable due to mixtures of unconventional styles
- Inconsistencies in user interface designs
- Long cycles of bug fixing, where the same bugs seem to return repeatedly
- Continuously slipping schedules with no clear causes
- Teams with high turnover, poor communication, and undisciplined work habits

There is no single fix for all these problems (just as they have no single cause), but one tool in the development manager's toolbox is the concept of managed, intentional code ownership. (By "code," I mean not only the source code but also all the artifacts that go

into defining, building, testing, and deploying a system.)

The four ownership models I present differ in how they grant one or more developers ownership of a system or subsystem (see Figure 1). The most common ownership model of the four (when there is any ownership) is subsystem ownership. Each developer takes primary responsibility for one or more subsystems. The product specialist and chief architect models grant ownership of an entire system to a single individual. In the latter case, several other developers contribute peripheral efforts. The last model, collective ownership, is a cornerstone of the newly popular development methodology, Extreme Programming.¹ Collective ownership maps the entire team to the entire system.

Product specialist

If a project is small enough for one individual to complete it in time to meet its business goals, then the product specialist model might be the best choice. The communication overhead to add even a second developer to a project can be significant. (Consider, however, the XP practice of pair programming on a truly two-person project, especially during system design and architectural definition.) Many software development organizations produce small products with lone developers and never suffer the overhead of large, dispersed teams. I have

A chief architect is an experienced developer who is competent from requirements to testing and has full responsibility for an entire system.

seen this approach applied most successfully on projects with technical complexity rather than size complexity.

The product specialist model's advantages are

- A single key developer might be more effective in carrying out all tasks consistently and to completion.
- Low communication overhead is required. External communications occur through a single channel.
- A clear and single-minded vision is easy to achieve.
- Iterative, incremental development is straightforward.

This model's disadvantages are

- It might be necessary to trade overall development time for development efficiency.
- The system will have a low *truck number*—only one developer need be hit by a truck to kill the project.
- Rogue individuals might “own” their code to the exclusion of organizational goals.

The biggest downside to having only one developer per project is fairly obvious: the size of system that one individual can manage is severely limited.

Subsystem ownership

Many teams grow from single-developer-sized systems to larger systems, divided into subsystems owned by the former product specialists. An unfortunate detour through code non-ownership is common. Once subsystems are defined well enough to assign owners, each is given to a single developer to implement according to interfaces negotiated with the owners of other nearby subsystems. The result is much less chaotic than non-ownership.

Subsystem ownership has several advantages:

- It ensures a consistent philosophy, vision, and style for each subsystem.
- It can easily assign responsibility for tasks and problems.
- The opportunity for specialization is significant.
- Scalability to larger projects is easiest.

Its disadvantages include

- Greater effort must be spent on communication to define interfaces between subsystems.
- There is still a low truck number per subsystem; the cost of turnover can be high because concentrated knowledge is lost.
- The overall system might be only as strong as its weakest subsystem and therefore only as good as what the weakest team member can produce.

A subsystem ownership model requires stable subsystem interfaces. Therefore, changes across subsystems are slowed by a combination of meetings, negotiation, and formal change requests. A danger of strongly enforced subsystem ownership is Conway's Law: that a system's architecture will mimic the structure of the organization that built it (see the “Conway's Law” sidebar).

Chief architect

The surgical team model Harlan Mills² and Terry Baker³ articulated and Fred Brooks summarized in *The Mythical Man-Month*⁴ has seen only modest use since it was proposed three decades ago. This is a shame because the advantages of a chief programmer-centered team are significant. Focus on the system as a whole leads to consistency, single-minded purpose, opportunities for global rather than local optimizations, and conciseness in representing and communicating the system to others.

Today Brooks' surgeon is more likely to go by the name I have used: chief architect. A chief architect is an experienced developer who is competent from requirements to testing and has full responsibility for an entire system. He or she concentrates on a system's architecture—taking a “big-chunk” view of the system as a whole—reviewing its layers, packages, components, third-party libraries, and system services. The proper role of a hands-on software architect is to define subsystems and their interfaces, create proof-of-principle evolutionary prototypes, and demonstrate common architectural mechanisms or patterns from design through sample code.

This model's advantages are

- Developers follow a consistent philosophy, vision, and style, and the chief ar-

chitect introduces and directs changes over time with predictable effect.

- Centralized knowledge reduces communication overhead—communication is mostly one to many rather than many to many.
- It better matches wide differences in developer skill.
- Opportunities for leadership and mentoring arise naturally.

The disadvantages are that

- The system truck number remains low.
- Assembling a team willing to support a single key individual is difficult.
- Achieving scalability to larger projects (more than eight people) is difficult.

Brooks' solution to the scalability problem is to begin a system's specification with an architect-led team before divvying each piece out to similarly organized programming teams.

Collective ownership

The basis of collective code ownership is that any developer can modify any subsystem. The difference between this and non-ownership is that team members ensure the integrity of the system as a whole as they make modifications. They write code within a common architecture with a common style and high quality to achieve a common purpose. Collective ownership is a lofty goal embodying altruism, positive team dynamics, good communication, and individual accountability.

Collective ownership is infeasible without several other XP practices related to source code quality during system implementation:

- Continuous integration
- 100 percent unit testing
- Strong coding style guidelines (see the "Homeowners' Associations" sidebar)

The practice of pair programming, also central to XP, might be another prerequisite. Certainly, frequent team member interactions and peer review are critical. Note that collective ownership is not merely permission to make widespread changes so long as they adhere to established architecture and style. It requires such practices because team members frequently rotate to new assignments (two- to three-week iterations in XP).

Conway's Law

Conway's Law is what happens when a team practices subsystem ownership at the beginning rather than at the end of a project. Conway's Law states that a system's architecture will mimic the organization that built it—in other words, if you have four groups working on a compiler, you'll get a four-pass compiler.¹⁻³

The problem with succumbing to Conway's Law is that old development-team communication patterns might not be the best choice for new software subsystem communication patterns. It's tempting to personify software and say, "the view layer talks to the model layer (just like I talk to Mike down the hall)," but the real reasons for separating model from view are technical. For example, the view layer is generally more platform specific, is more likely to change, and requires different kinds of testing. It is, however, too easy to find user interface specialists and business logic specialists rather than programmers or architects who understand how to structure a model to purposely anticipate view layer changes or how to make best use of both automated user interface testing and semi-automated low-level unit testing.

Another problem is that layers or subsystems are only one way to slice a software system—and not usually the best way from a project management and scheduling perspective. Schedules and business value are typically measured in functionality, use cases, or features. These are generally orthogonal to subsystems, layers, classes, or other lines of modularity. Teams with responsibilities distributed by subsystem are likely to require higher communication overhead to complete any given user feature because each feature crosses several layers.

The reasons Conway's Law is so powerful are not to be ignored. The stability of subsystems, the opportunity for specialization, and the establishment of developer communication habits are all positive effects. The trick is to get the architecture before the organization. Better yet, wait until the second or third revision of the architecture before freezing it in the organization. Let your team settle into one developer per subsystem by a project's end, but don't divide up by horizontal layers until significant vertical slices have been completed. Don't stabilize organizations until subsystems really are stable. And keep communication channels open and dispersed for as long as you can. With luck, you'll produce a focused team rather than merely coordinate a group of specialists.

References

1. F. Brooks, *The Mythical Man-Month*, anniversary ed., Addison-Wesley, Boston, 1995.
2. M.E. Conway, "How Do Committees Invent?" *Datamation*, vol. 14, no. 4, Apr. 1968, pp. 28-31.
3. J. Coplien, "Organization and Architecture," 1999 CHOOSE Forum on Object-Oriented Software Architecture, 1999; www1.bell-labs.com/user/cope/Talks/Arch/CHOOSE99/.

On the positive side, collective ownership

- Automatically distributes communication overhead over time and functionality.
- Encourages greater team interaction, cohesiveness, and creativity.
- Requires and enforces a consistent style and philosophy across the system. Code sharing or reuse is more likely than with subsystem ownership.
- Easily handles both team growth and shrinkage.

Homeowners' Associations

Coding style guidelines are to code ownership much as homeowners' associations are to home ownership—both a boon to neighborhood quality and a potential detriment to individuality. A suburban homeowners' association that can enforce a consistent quality of appearance and upkeep can result in a neighborhood with an improved sense of identity as a whole. The sports car fanatic at the end of the block is prevented from rebuilding his relic in the middle of the front yard, and the elderly lady with a green thumb next door happily shares her divided perennials with everyone on the street.

It's the same with coding style guidelines. The C++ macro maniac is prevented from littering his code with strange preprocessor tricks, while the team-oriented engineer who designs a reusable hardware access layer is encouraged to write a usage guide as a supplement to the organization's style guidelines.

Coding guidelines are an emotional topic (just like sports cars). Developers complain about picayune syntax details (the braces are never in the right place) and the loss of creativity. However, picture the house across the street painted "creative" pink with white trim and candy red shutters. In isolation, it's a cute little house. In a beige and brown neighborhood, it's at best an amusing anachronism.

Most developers who have lived in a neighborhood or two have seen code that could be described as "at best an amusing anachronism." A good coding style guideline is the objective measure for avoiding software inconsistency. It takes effort to draft (and regularly redraft) a programming guide that goes beyond brace placement and parameter naming, that encourages quality of both appearance and execution, and that is tailored to a specific team and project's needs.

Here is some advice to definers of coding style: before defining the coding style, first define the organizational style. Define the individuality of the team before surrendering the individuality of its members. If everyone in a neighborhood likes to rebuild sports cars on the front lawn, then a homeowners' association could actually encourage them and make the neighborhood a better place by building a sense of sports car community. Write your coding style to match and sustain your team building style.

Don't ignore the effort needed to review code against the guideline. Whether it's through pair programming or formal inspections, the discussion must go beyond naming conventions and macro usage to consider correctness, adherence to the system's overall architecture, suitability in comparison to alternatives, and long-term maintainability. The result is a neighborhood with well-kept lawns (or happy sports car fans, as the case may be), properly maintained homes, and positive relationships between neighbors.

(By the way, the opening brace belongs on a new line.)

- Rapidly and effectively responds to incremental or changing requirements.

Its disadvantages are

- An unorthodox organizational and scheduling model might be difficult to establish and manage.
- Responsibility for tasks or problems might be difficult to establish.
- Scalability is limited to 10 to 15 developers.
- There is the potential for "Brownian mo-

tion" in the code—repeated changes according to differing personal styles.

- The system might be missing an overall skeleton or architecture—an initial direction and unity of purpose might be difficult to achieve.

Dynamic code ownership

Because each model has different strengths and weaknesses, the short answer to the question, "Which should we choose?" is "all of them." Consider a project in terms of phases from the iterative Unified Process:⁵ Inception, Elaboration, Construction, and Transition. During Inception, the main goal is to gather requirements and define the process for building the product. During Elaboration, goals include risk reduction, evolutionary and exploratory prototyping, and the establishment of system patterns, metaphors, mechanisms, and skeletal structure. Construction focuses on programming to complete functionality, and Transition proceeds through final testing, deployment, and release.

You can address these changing needs by intentionally changing the system ownership model over its lifetime. In the early stages, use the product specialist model and select an individual (often called a product manager) to articulate the vision for a new or revised system. The product manager develops the system's business case, high-level features, and core requirements. This individual increasingly accepts the assistance of systems analysts to gather the requirements but remains the project's sole proprietor until those requirements begin to fill in.

As the system definition grows too big for one individual, shift to a chief architect model. The architect might initially help expand the requirements but should soon turn his or her attention to whole system design. A small and growing team around the architect performs experiments directed at reducing unknowns in the system, prototyping user interfaces, establishing the tools and environment, and performing the planning and process engineering needed to put the project on a fast track.

Once the architecture is well defined, the number of developers usually increases to fit the amount of work to be done. Here is the opportunity to practice collective ownership to propagate the established architecture to a growing team of programmers. The ad-

Phase goal	Vision	Architecture	Functionality	Stability
Code ownership model	Product specialist	Chief architect	Collective ownership	Subsystem ownership
Development phase	Inception	Elaboration	Construction	Transition

Figure 2. Vary the code ownership model over the phases of a project to fit each phase's purpose.

vantages of collective ownership for rapidly adding new features with minimal communication overhead directly support the Construction phase goal of building a quality system as rapidly as possible.

Finally, once subsystems are deemed stable (as Construction approaches the Transition phase), assign specific owners to finalize them and to control their changes in a project's last stages.

Figure 2 shows the transition in code ownership over time superimposed on the development phases and their goals. Note that the ownership transitions do not occur overnight. This is possible because of changing team size. For example, the transition from lone product specialist to chief architect near the end of Inception occurs over time. The product specialist retains ownership over the requirements documents while the newly added chief architect and a growing team of developers take ownership of design models, prototypes, and associated development tools. Next, as Elaboration nears its end, newly added programmers take collective ownership of existing subsystems while the chief architect retains ownership of still-to-be-added subsystems and key interfaces. Moving into Transition, the team ideally shrinks and remaining team members take specific responsibility for the final polishing of nearly completed subsystems.

Figure 3 shows the variation of team size over a project's lifetime. This diagram's front half is familiar. Many projects rapidly add developers when racing toward the deadline. Fewer projects take advantage of the benefits that the back half of the diagram implies. A planned shrinkage of the team with a coincident increase in subsystem ownership results in a software system that stabilizes prior to its release, exactly what is needed to "get it out the door."

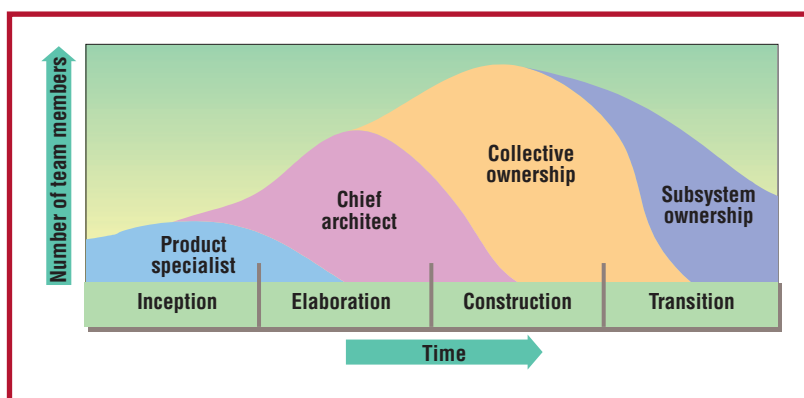


Figure 3. Ramp team size both up and down to best match a project's phases.

How can a project benefit from both the ramp up and ramp down without high staff turnover or idle developers? Figure 4 contains one answer. Divide the overall team into two parts assigned either to different products or different releases of the same product. With this approach, once a chief architect has given up a benevolent dictatorship to a collective republic of programmers, it is precisely the time to take up the same role on the next release or a new initiative. A continually remixing pool of programmers gives up old subsystem ownership for new system collective ownership, then repeats the cycle.

A beneficial effect of this approach for long-established organizations is that a team is not reassigned en masse to a new project or new release. Each new effort starts small with a uniform vision and precisely crafted architecture. There is no possibility of design by committee, "sort of like we did the last project."

This solution to the problem of varying staff size is not without its own problems. Even partly overlapping normally sequential releases leads to greater risk, lower feedback, and potentially conflicting aims within the two halves of the whole team. Another difficulty is matching the duration of release cycles to sustain the pattern of Figure 4 over more than two cycles. This is even more diffi-

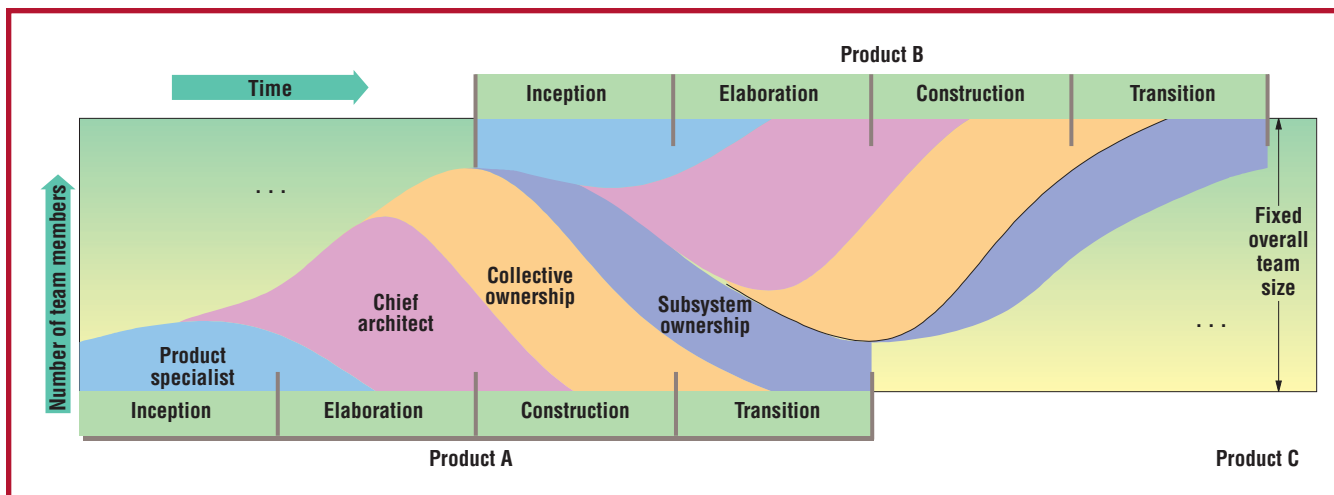


Figure 4. Schedule parallel projects that ramp up and down to maintain a total overall team size.

cult when different products must be matched in life cycle. It takes extra management to shift features between releases to not only meet deadlines but also to synchronize deadlines between projects or project phases.

Scalable code ownership

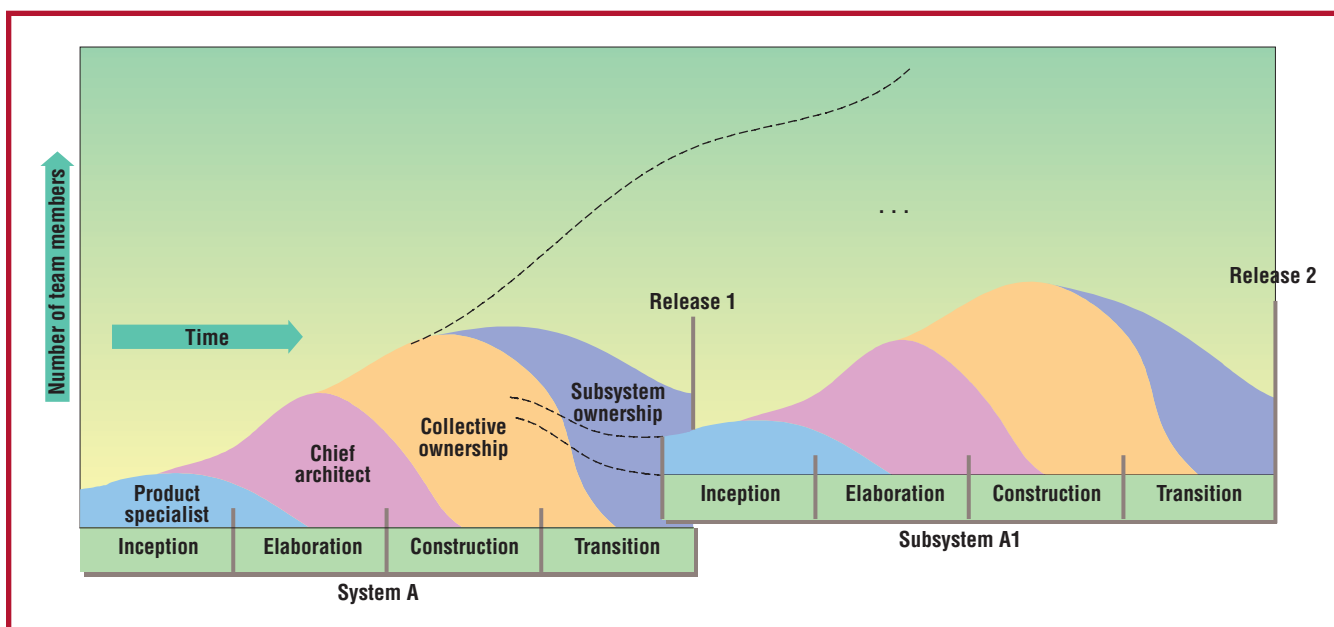
What if a system grows so large that today's subsystem is tomorrow's whole system? The developer who is assigned ownership of a release 1 subsystem might well become the lone caretaker of that subsystem-turned-system in release 2. Figure 5 shows starting a whole new development and code ownership cycle out of each subsystem heading into a second release. With a sufficiently skilled and growing team, this model could be sustained for several releases. Note that "releases" might be evolutionary milestones on the way from a proof-of-principle prototype to a large finished

system. In fact, the dynamic code ownership model might be best suited to projects that are not only iterative but also incremental.

For a modest-sized system, Figure 4's resource-leveling scheme is easier to adjust with shorter and smaller release increments. Small release increments also make it easier to adjust the subsystem-to-system growth model to match actual growth in team size. On the other hand, the increments must be large enough to justify dynamic code ownership in the first place. There is much room for skilled software engineering management.

The evidence for this approach's viability, as with too many aspects of software engineering, lies in the negative examples. I have been a secondary participant in a multimillion-dollar failure that I attribute largely to a lack of ownership. The project had a requirements set and an architecture that seemed sufficient to make it successful, but too many

Figure 5. Grow a large system by treating each subsystem as a separate system in subsequent release cycles.



About the Author



Martin E. Nordberg III is a senior software architect working as a consultant with Blueprint Technologies. His career has steadily moved from software that is technically complex (computational fluid mechanics) to software that is organizationally complex (large distributed information systems). His interests include programming language details, object-oriented analysis and design, design patterns, software architecture, and the ripening field of aspect-oriented programming and design. He received his BS and MBA degrees from Rensselaer Polytechnic Institute and his PhD in chemical engineering from the University of Massachusetts. Contact him at Blueprint Technologies, 8618 Westwood Center Dr., Ste. 310, Vienna, VA 22182; martin.nordberg@acm.org.

team members took too little ownership of realizing the design in a high-quality, incrementally delivered, working system. The team moved from non-ownership to half-hearted subsystem ownership, never achieving a cohesive whole-system vision. Worse still, the project ignored the implications shown in Figure 5 and attempted to jump from a two-person-year stopgap prototype to a complex 50-person-year final system without first releasing a working 10-person-year intermediate system.

You might have experienced an unhappy project or two in your own past. Would attention to code ownership, dynamically adjusted over the project's life, have made a difference? There are, of course, myriad factors that contribute to successful software development projects.⁶ I have offered one area where there is room for improvement. Dynamic code ownership might well put your team on track to deliver high-quality software systems. 

References

1. K. Beck, *Extreme Programming Explained*, Addison-Wesley, Boston, 2000.
2. H. Mills, "Chief Programmer Teams, Principles, and Procedures," IBM Federal Systems Division Report FSC 71-5108, IBM, Gaithersburg, Md., 1971.
3. T. Baker, "Chief Programmer Team Management of Production Programming," *IBM Systems J.*, vol. 11, no. 1, Jan. 1972, pp. 56-73.
4. F. Brooks, *The Mythical Man-Month*, anniversary ed., Addison-Wesley, Boston, 1995.
5. I. Jacobson, G. Booch, and J. Rumbaugh, *The Unified Software Development Process*, Addison-Wesley, Boston, 1999.
6. R.H. Thayer, ed., *Software Engineering Project Management*, 2nd ed., IEEE CS Press, Los Alamitos, Calif., 1997.

For more information on this or any other computing topic, please visit our Digital Library at <http://computer.org/publications/dlib>.

BOOK REVIEWERS

Are you looking to
contribute but don't have
much time?

Why not review a book for DS Online?
You'd be volunteering your services to a
great publication, gaining knowledge
(and getting a free book!).

Contact DS Online's book review editor
Marcin Paprzycki at
marcin@a.cs.okstate.edu

IEEE
distributed systems
ONLINE
Expert-authored articles and resources